Computação Escalável (2COP022)

Larissa C. Shimomura

Aula 8- 09/10

Avisos

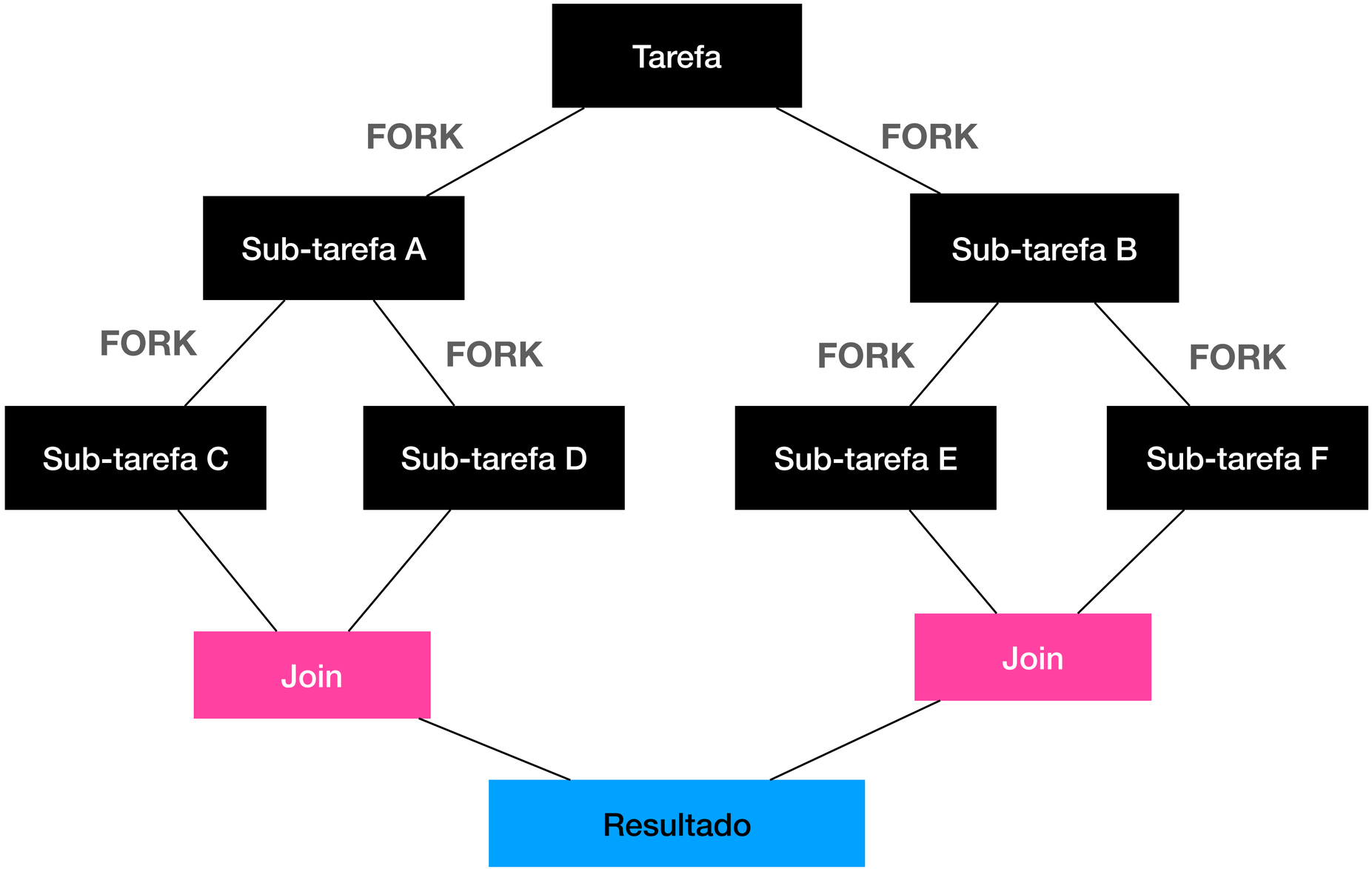
- Aulas de Projeto Integrado de IA dos dias 20 e 21 de Outubro serão de Computação Escalável em reposição a aula passada.

Java ForkJoin

Dividir para Conquistar

- **Definição:** O ForkJoinPool é um framework voltado para acelerar tarefas pesadas que podem ser **quebradas em subtarefas menores** e executadas em paralelo.
- **Como Funciona:**
  - **Fork (Dividir):** Quebra uma tarefa grande em tarefas menores de forma recursiva até que fiquem simples o suficiente.
  - **Join (Juntar):** Aguarda a conclusão de cada subtarefa e combina seus resultados para formar a resposta final.
- **O Diferencial (Work-Stealing Algorithm):**
  - Se a thread A terminar suas tarefas antes da thread B, a thread A **"rouba" trabalho** do final da fila da thread B.

Java ForkJoin - Visualmente



Java - ForkJoinPool

Classes Principais:

- ForkJoinPool: O gerenciador de threads/execução.
- ForkJoinTask: A tarefa a ser invocada no ForkJoin que podem ser implementadas como:
    - RecursiveTask<V>: Usado para tarefas que **retornam** um valor/resultado.
    - RecursiveAction: Usado para tarefas que **não retornam** valor (void).
    - Deve-ser implementar a lógica da tarefa no método compute().

Exemplo - Busca maior valor em um vetor

```java
public class BuscaMaiorValorTask extends RecursiveTask<Integer> {
    private static final int LIMIAR = 500;
    int[] array;
    inicio;
    fim;

    rTask(int[] array, int inicio, int fim) {
        y;
        cio;
```

```java
@Override
protected Integer compute() {
    //Caso Base (Processamento Sequencial)
    if ((fim - inicio) <= LIMIAR) {
        int max = array[inicio];
        for (int i = inicio + 1; i < fim; i++) {
            if (array[i] > max) {
                max = array[i];
            }
        }
        return max;
    }
    // Divisão em Subtarefas
    int meio = inicio + (fim - inicio) / 2;
    BuscaMaiorValorTask esquerda = new BuscaMaiorValorTask(array, inicio, meio);
    BuscaMaiorValorTask direita = new BuscaMaiorValorTask(array, meio, fim);
    // Dispara a tarefa da esquerda assincronamente
    esquerda.fork();
    // Processa a tarefa da direita na thread atual
    int maxDireita = direita.compute();
    // Aguarda a tarefa da esquerda e combina os resultados
    int maxEsquerda = esquerda.join();
    return Math.max(maxEsquerda, maxDireita);
}
```

Implementa a tarefa que você está executando de forma parecida com uma recursão: 1. Implementação do caso base

2. Realiza a divisão das tarefas - direita e esquerda

3. Fork para disparar uma subtarefa de forma assíncrona

4. join() para espera a tarefa terminar e o Math.max para retornar o maior valor

Exemplo - Busca maior valor em um vetor

Common Pool - um pool de threads comum para aplicação Java gerenciado pela JVM

```java
public class BuscaMaiorValorTask extends RecursiveTask<Integer> {
    private static final int LIMIAR = 500;
    private final int[] array;
    private final int inicio;
    private final int fim;

    public BuscaMaiorValorTask(int[] array, int inicio, int fim) {
        this.array = array;
```

```java
public static void main(String[] args) {
    // Criando dados de teste
    int[] dados = new int[50_000_000];
    for (int i = 0; i < dados.length; i++) {
        dados[i] = ThreadLocalRandom.current().nextInt(1, 100_000_000);
    }
    // Execução no ForkJoinPool
    ForkJoinPool pool = ForkJoinPool.commonPool();
    BuscaMaiorValorTask tarefaPrincipal = new BuscaMaiorValorTa...th);
    long tempoInicio = System.currentTimeMillis();
    int maiorValor = pool.invoke(tarefaPrincipal);
    long tempoFim = System.currentTimeMillis();
    System.out.println("Maior valor encontrado: " + maiorValor);
    System.out.println("Tempo de execução: " + (tempoFim - tempoInicio) + " ms");
}
```

Utiliza o método invoke para iniciar/invocar o método principal

Exercício 1

Busca em vetor

- Dado um vetor [1,70,40,20,89,34,71,56,2,6,30,23], escreva um programa que utiliza 3 threads (além da thread principal) que calcula a soma dos valores deste vetor utilizando ExecutorService e a interface Callable do Java.

- Escreva o mesmo programa agora utilizando o ForkJoin em Java.

Exercício 2

- Implemente a soma de todos os elementos de uma matriz, utilizando decomposição de dados em blocos e ForkJoin.
- Dada uma árvore, conte quantos nós ela possui utilizando Fork/Join.

Arquivos Distribuídos e Introdução a Big Data

Introdução a Big Data

- Até agora vimos como programar em um único nó utilizando Java….mas se os dados forem grandes demais e não couberem na memória?
- Idealmente nós utilizaríamos um sistema distribuído onde cada nó pode conter parte dos dados…
- Este é uma forma bastante utilizada em soluções de Big Data

Introdução a Big Data

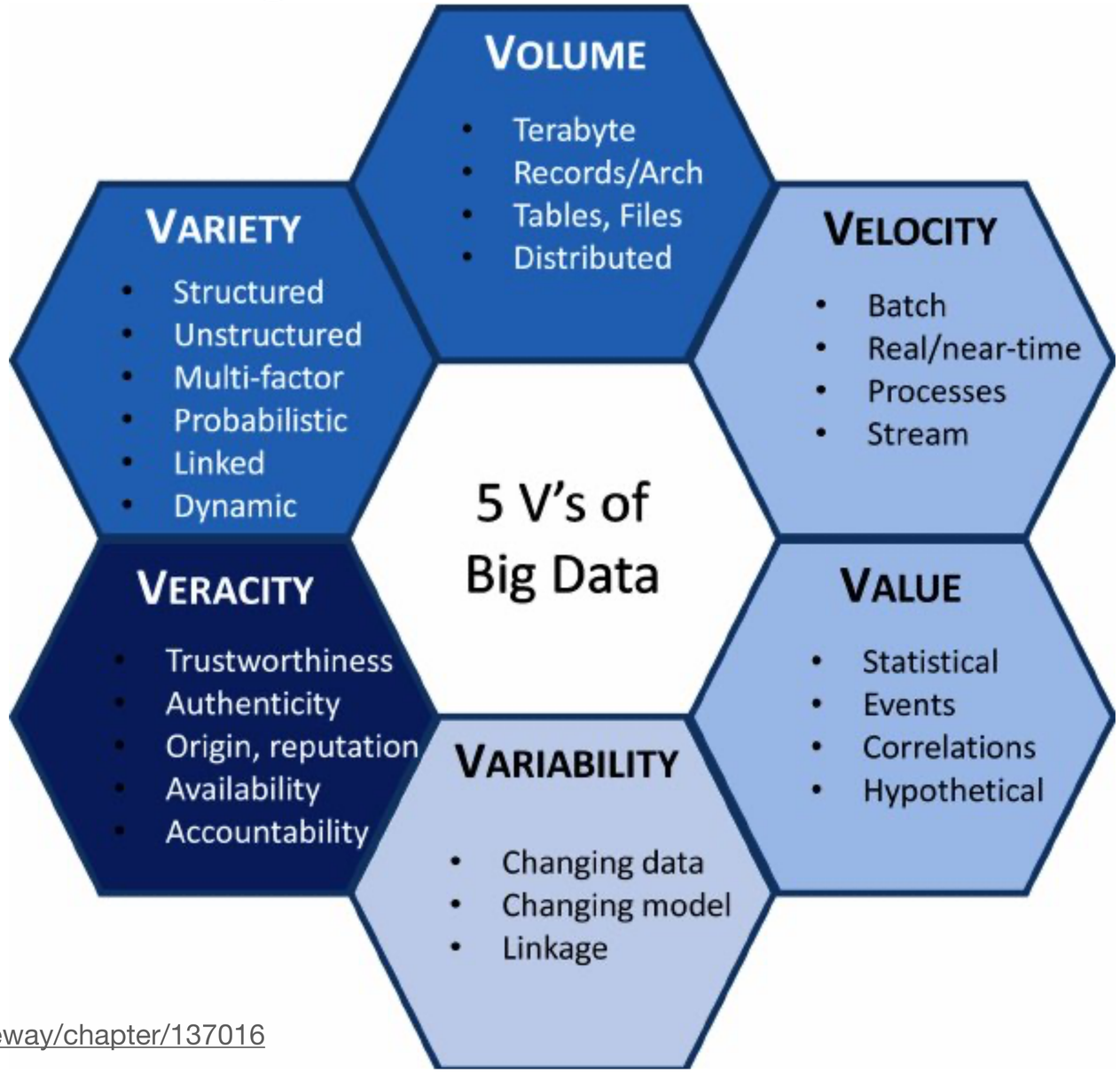
- ˝Simplificando, big data é um conjunto de dados maior e mais complexo, especialmente de novas fontes de dados. Esses conjuntos de dados são tão volumosos que o software tradicional de processamento de dados simplesmente não consegue gerenciá-los. No entanto, esses grandes volumes de dados podem ser usados para resolver problemas de negócios que você não conseguiria resolver antes.˝

- Retirado de Oracle Brasil (https://www.oracle.com/br/big-data/what-is-big-data/)

Introdução a Big Data - 5 V´s

- Big Data geralmente é caracterizado pelos 5 V´s:
  - **V**olume - Quantidade massiva de dados gerados a todo momento
  - **V**elocidade - Rapidez extrema que novos dados são produzidos
  - **V**ariedade - Diversidade de dados, desde estruturados (tabelas SQL) até não estruturados (textos, áudios, vídeos e logs).
  - **V**eracidade - O grau de confiabilidade, precisão e qualidade dos dados coletados, eliminando ruídos e informações falsas.
  - **V**alor - A capacidade de transformar esse grande volume de dados tratados em insights práticos e vantagem competitiva para o negócio.

Introdução a Big Data - 5 V´s

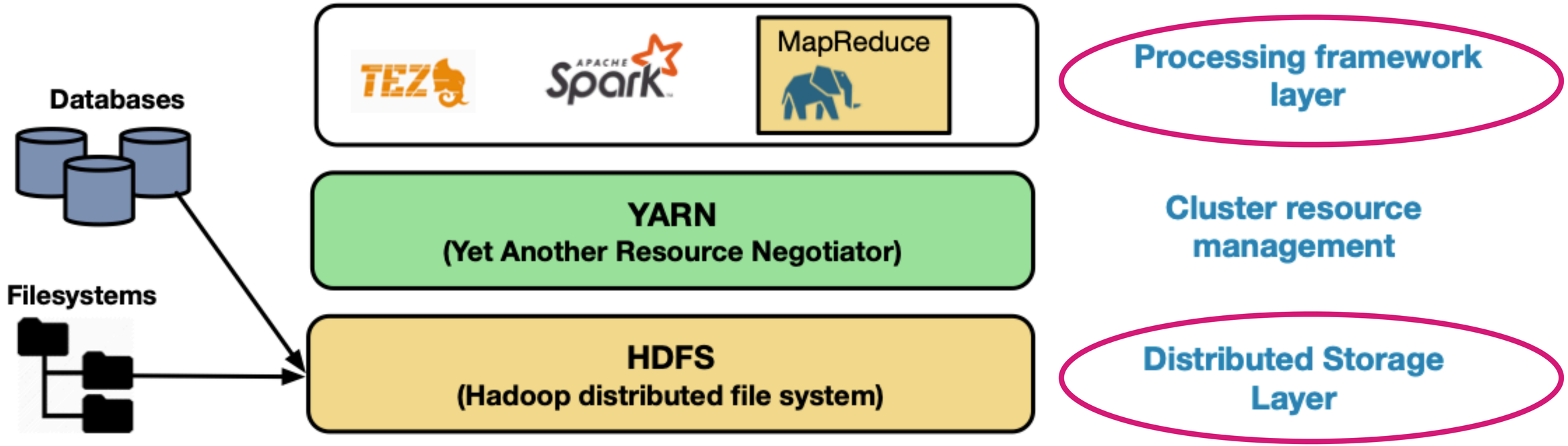


Retirado de https://www.igi-global.com/gateway/chapter/137016

Sistema Hadoop

- Framework para processamento em grande escala
- **Agrupa computadores ˝baratos˝ (*Commodity Hardware*):** Roda em máquinas simples de mercado.
- **Tolerância a Falhas:** Se um computador do cluster queimar no meio do trabalho, o Hadoop tem como pegar a cópia do dado e continuar a tarefa em outra máquina sem interromper nada.
- **Levar o Código aos Dado (*Data Locality*):** Em vez de transferir Terabytes de dados pela rede até o seu programa, o código é executado *dentro* do servidor onde o dado já está guardado.
- **Cluster com arquitetura shared-nothing: Cada máquina tem sua própria CPU, hardware etc.**

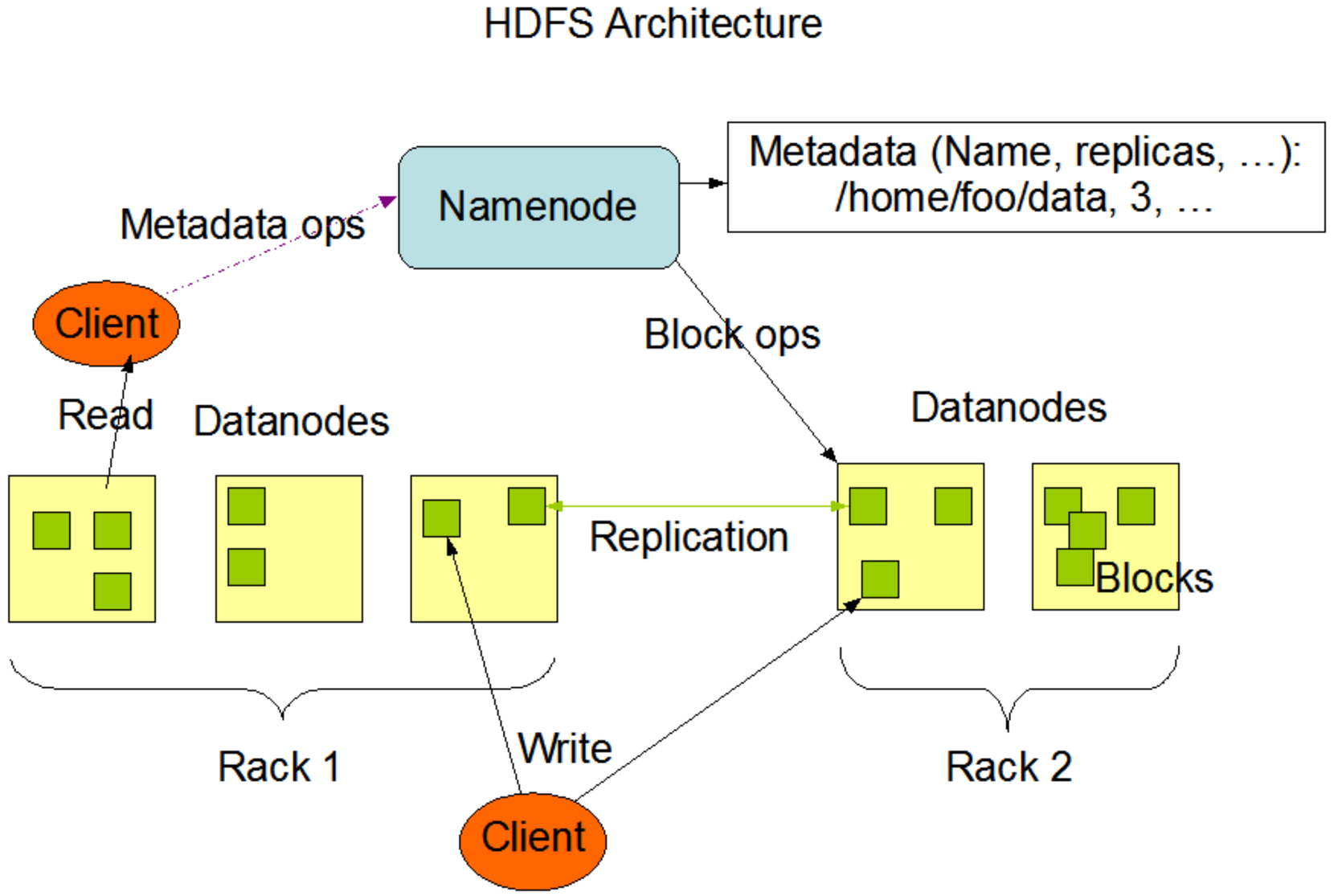
Framework Hadoop - Componentes



Retirado de https://gquercini.github.io/courses/big-data-marseille/big-data-2.pdf

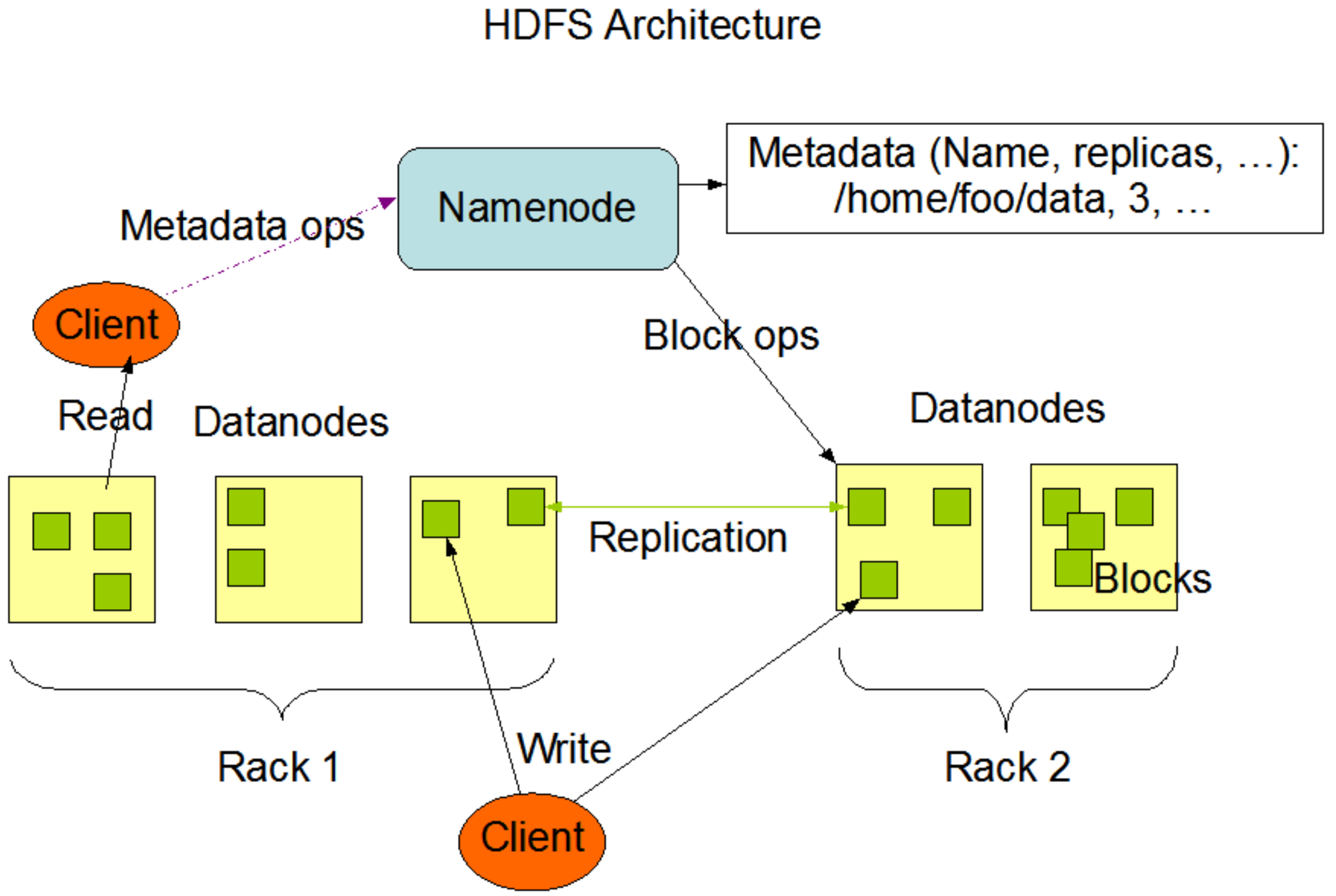
Hadoop Distributed File System

- Arquitetura baseada em mestre/escravo (master/ slave)
- Cada cluster HDFS tem um Namenode —> mestre
- Mestre gerencia o acesso aos arquivos, é ele que abre, fecha, renomeia arquivos e etc.



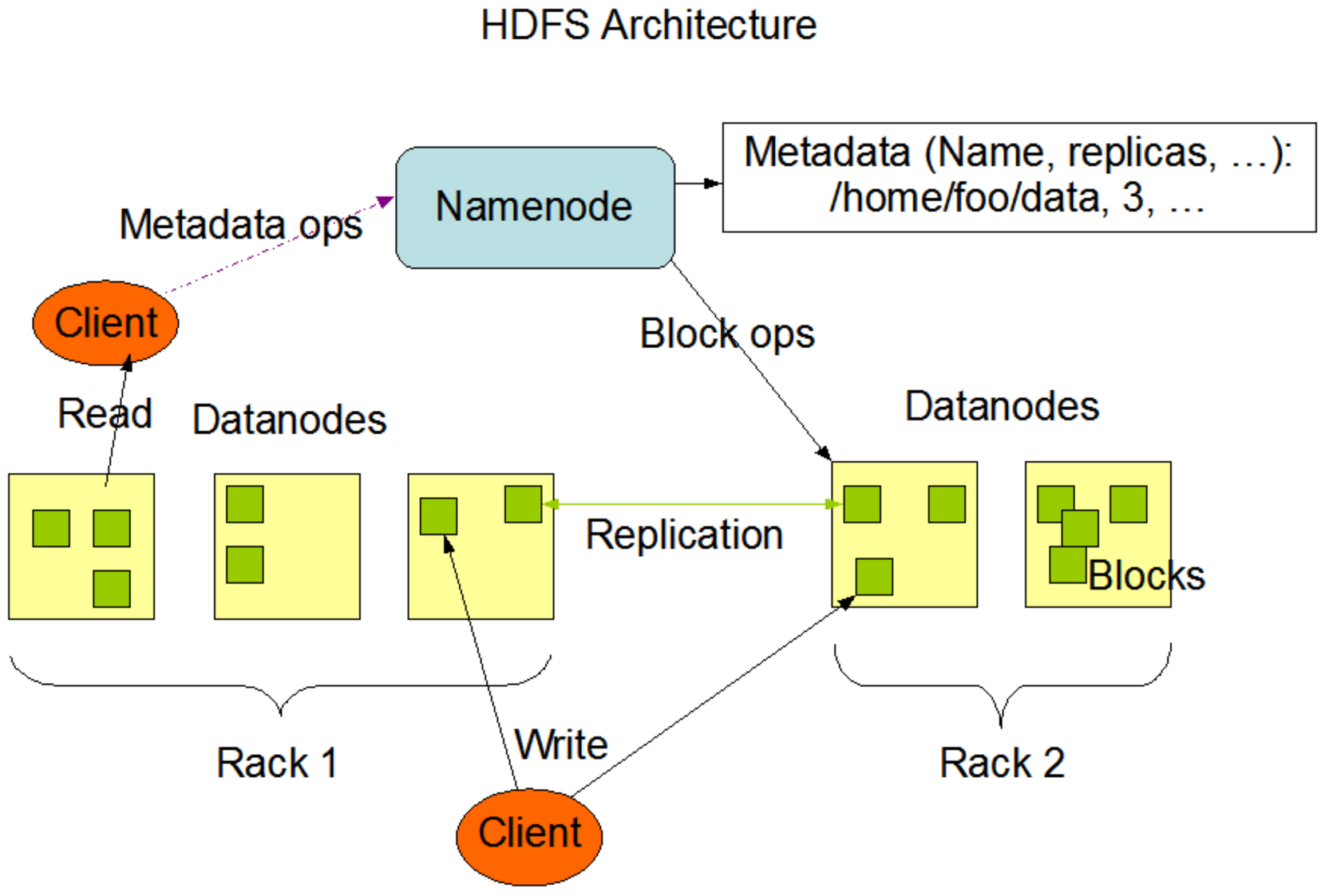
Hadoop Distributed File System

- Arquitetura baseada em mestre/escravo (master/slave)
- Cada nó do HDFS tem pelo menos um Datanodes —> escravo
- Datanode ele gerencia os dados que estão armazenados naquele nó
- Ele que gerencia a leitura e a escrita àqueles dados quando requisitados pelo cliente.



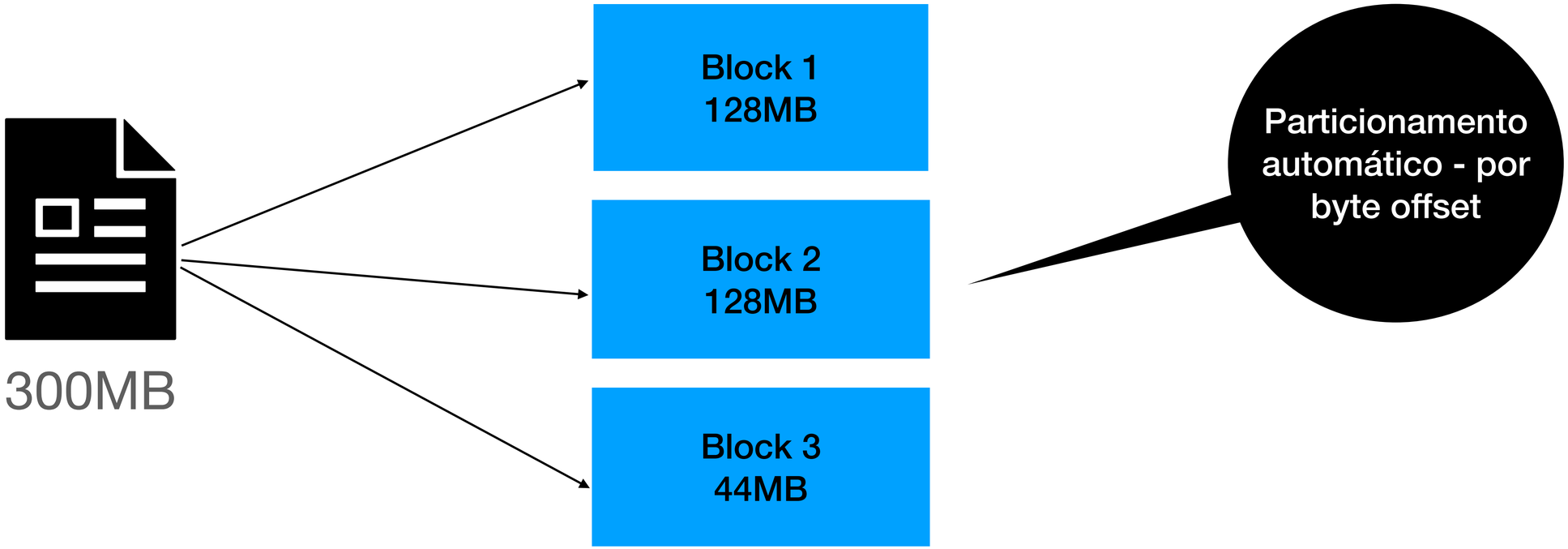
Hadoop Distributed File System

- Internamente o arquivo é dividido em um ou mais blocos que são armazenados em um conjunto de Datanodes.
- O Namenode que determina o mapeamento dos blocos para cada um dos Datanodes
- Namenodes e Datanodes foram implementados em Java —> fácil portabilidade

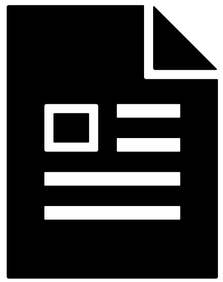


Hadoop Distributed File System

- O HDFS utiliza particionamento em blocos de tamanho fixo, o padrão é 128MB.
  - Pode ser alterado como parâmetro
  - Atenção: aqui estamos falando de particionamento dos dados a nível de infraestrutura.



Hadoop Distributed File System



300MB

Namenode

/file/part0-1.. -> B1, B2
/file/part0-2.. -> B2, B3
/file/part0-3.. -> B3, B1

Datanode 1

Bloco B1
128MB

Datanode 2

Bloco B2
128MB

Datanode 3

Bloco B3
44MB

HDFS - Replicação

- Cada Bloco é replicado em uma ou mais máquinas do cluster
- O número de réplicas é chamado de fator de replicação - o padrão é 3.
- 2 réplicas do mesmo bloco nunca são armazenadas em um mesmo DataNode.
- Quando possível, 2 blocos do mesmo arquivo são armazenados em um mesmo DataNode.

HDFS - Notas sobre o particionamento

- É possível implementar um método de particionamento lógico sobre o Hadoop
- Porém o armazenamento físico dos dados continua sendo em blocos e o Hadoop utiliza ponteiros para seguir o particionamento lógico
- Para ˝forçar˝ o particionamento físico seguir o mais próximo do lógico podemos:
  - Utilizar pastas e diretórios
  - Utilizar outro tipo de serialização de arquivos, e.g. o Parquet.

Hadoop - Map-Reduce

- Modelo de programação e processamento distribuído.
- O MapReduce processa os blocos no próprio local/nó em que estão armazenados
  - Evita o tráfego de grandes volumes de dados pela rede
  - Localidade de Dados
- Ideia central baseada em **Dividir para Conquistar**: em vez de carregar um arquivo gigante na memória de uma única máquina, a tarefa é quebrada em centenas de pequenas tarefas paralelas.

Hadoop - Map-Reduce

- Tem principalmente 2 fases:
  - Map -> mapeia uma tarefas aos blocos de dados **(Dividir)**
  - Reduce -> reduz os resultados a fim de obter o resultado final **(Conquistar)**
  - Programador precisa apenas implementar as tarefas de Map e Reduce
  - A fase Reduce é opcional, muitas vezes apenas Map é utilizado.
- Existe uma fase intermediária de shuffle/ sort entre as duas fases.

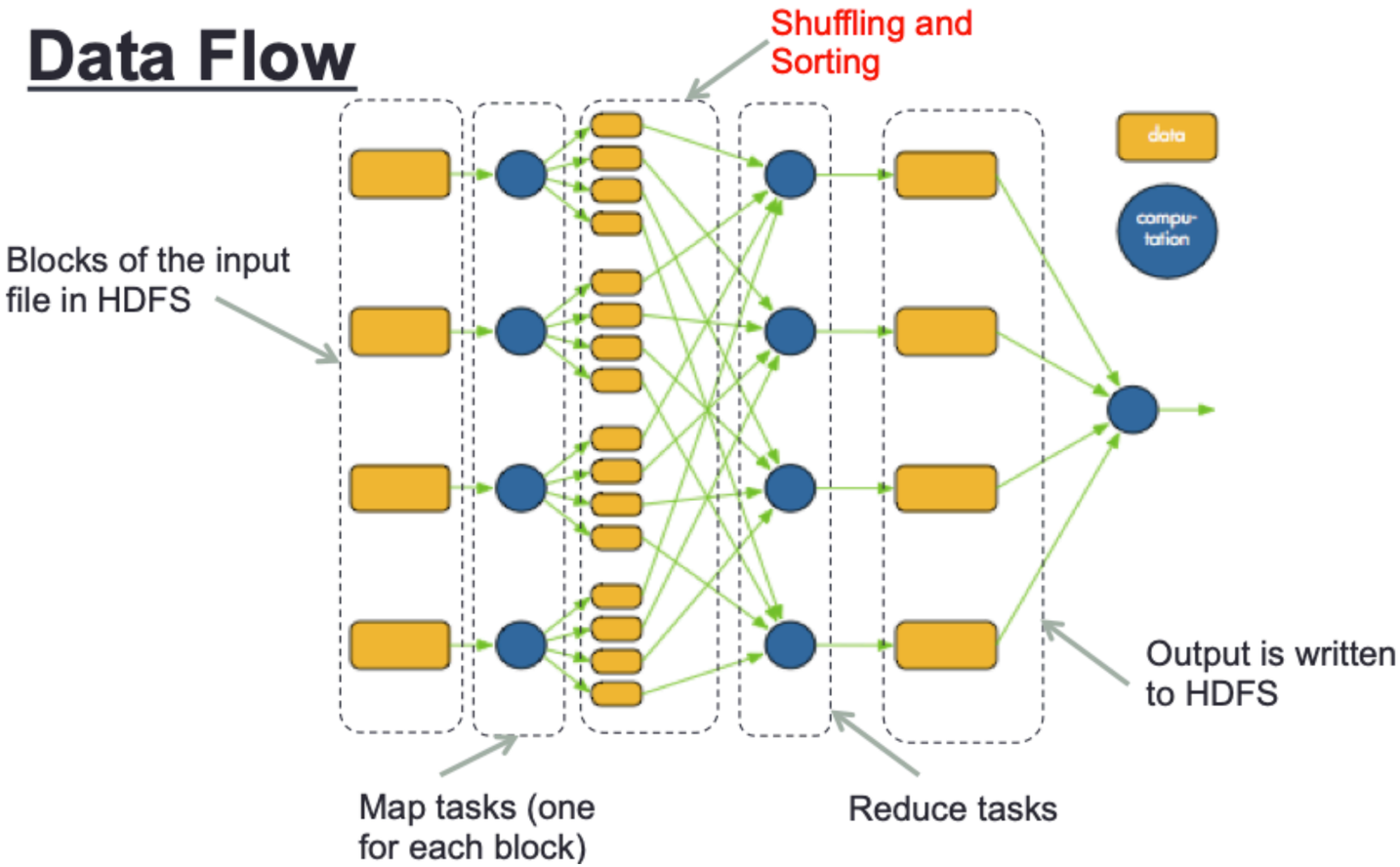
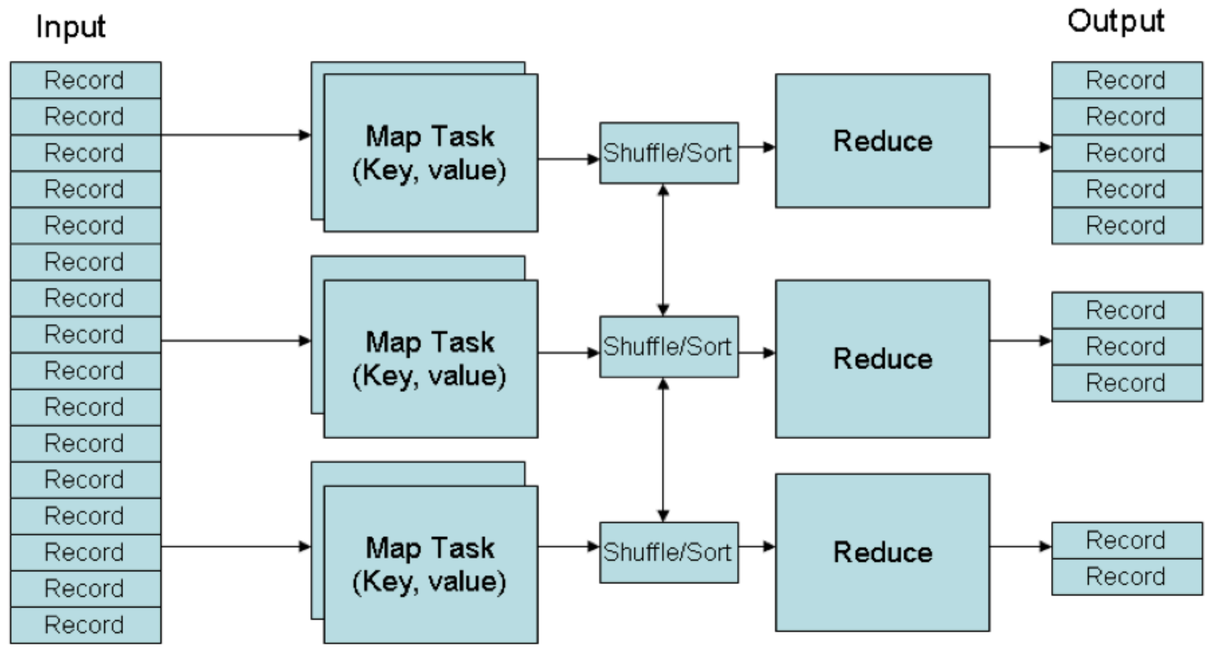


Imagem retirada de https://web.cs.wpi.edu/~cs561/s12/Lectures/6/Hadoop.pdf

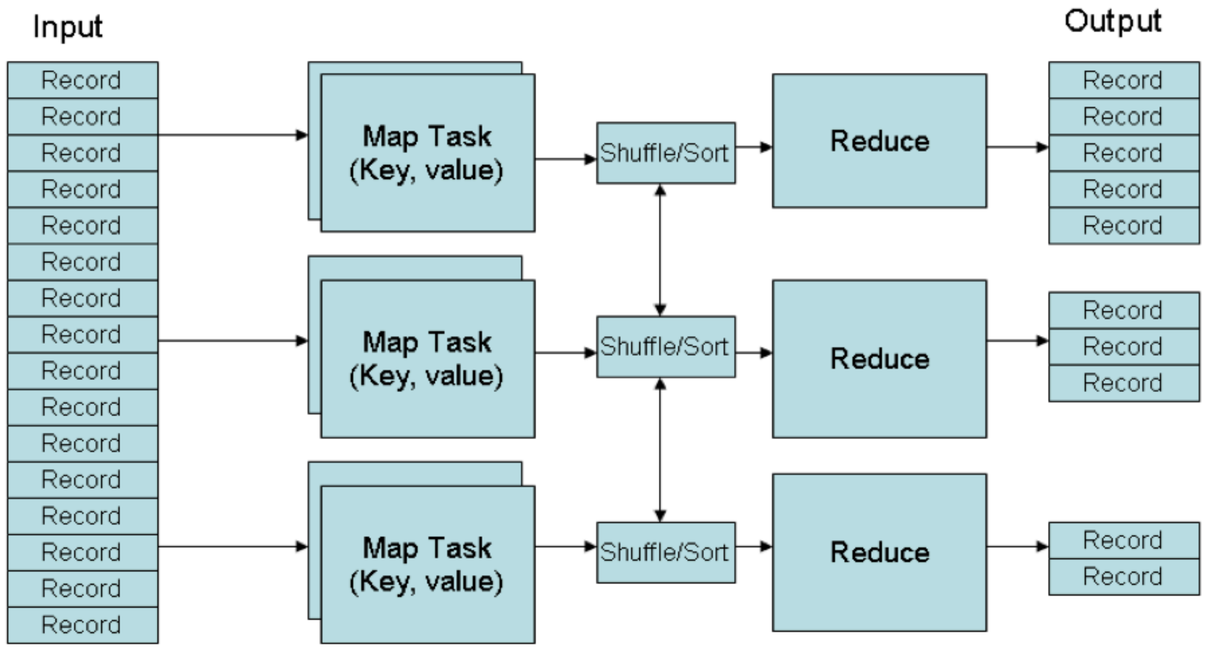
Hadoop - Map-Reduce

- As funções de map e reduce consomem e produzem pares (chave, valor)
- Os programadores que definem os tipos de chave e os tipos de valores
- É aplicada a função/tarefa implementada no map sobre um bloco.
- O output são pares de (chave, valor)



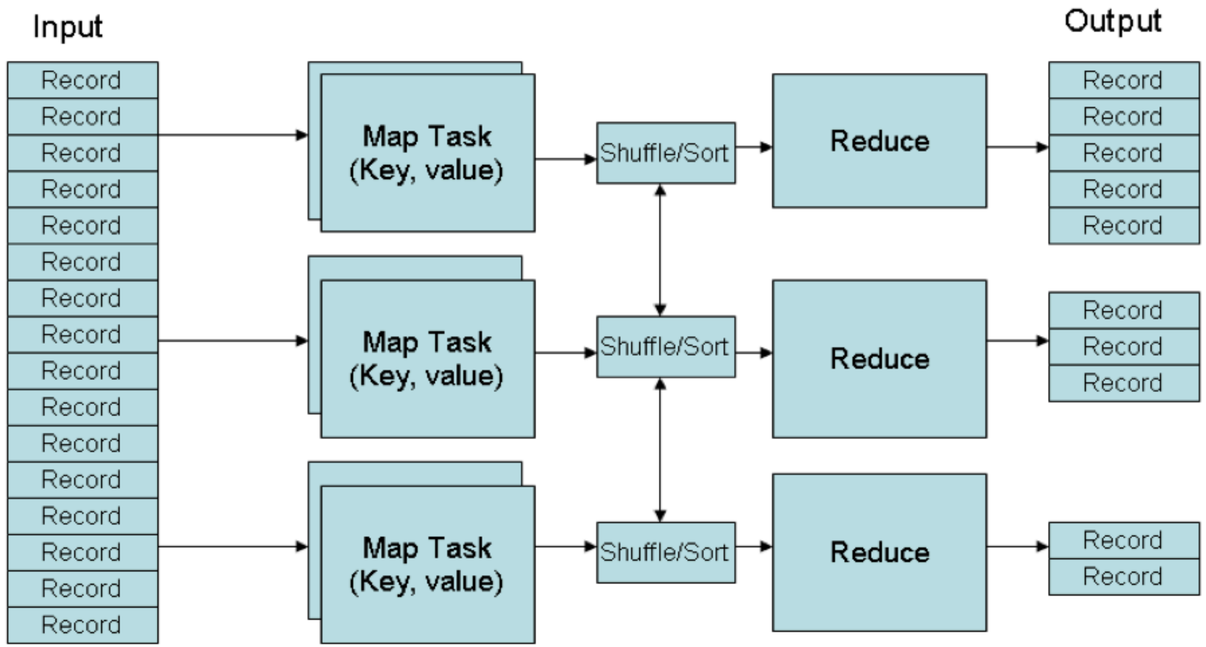
Hadoop - Map-Reduce

- É aplicada a função/tarefa implementada no map sobre um bloco.
- O output são pares de (chave, valor)
- Neste ponto ocorre uma operação de ***shuffle,*** onde as chaves são reorganizadas de modo que as mesma chaves são enviadas para o mesmo **reducer**
- Estas chaves podem ser agrupadas ou ordenadas em cada reducer e então são processadas



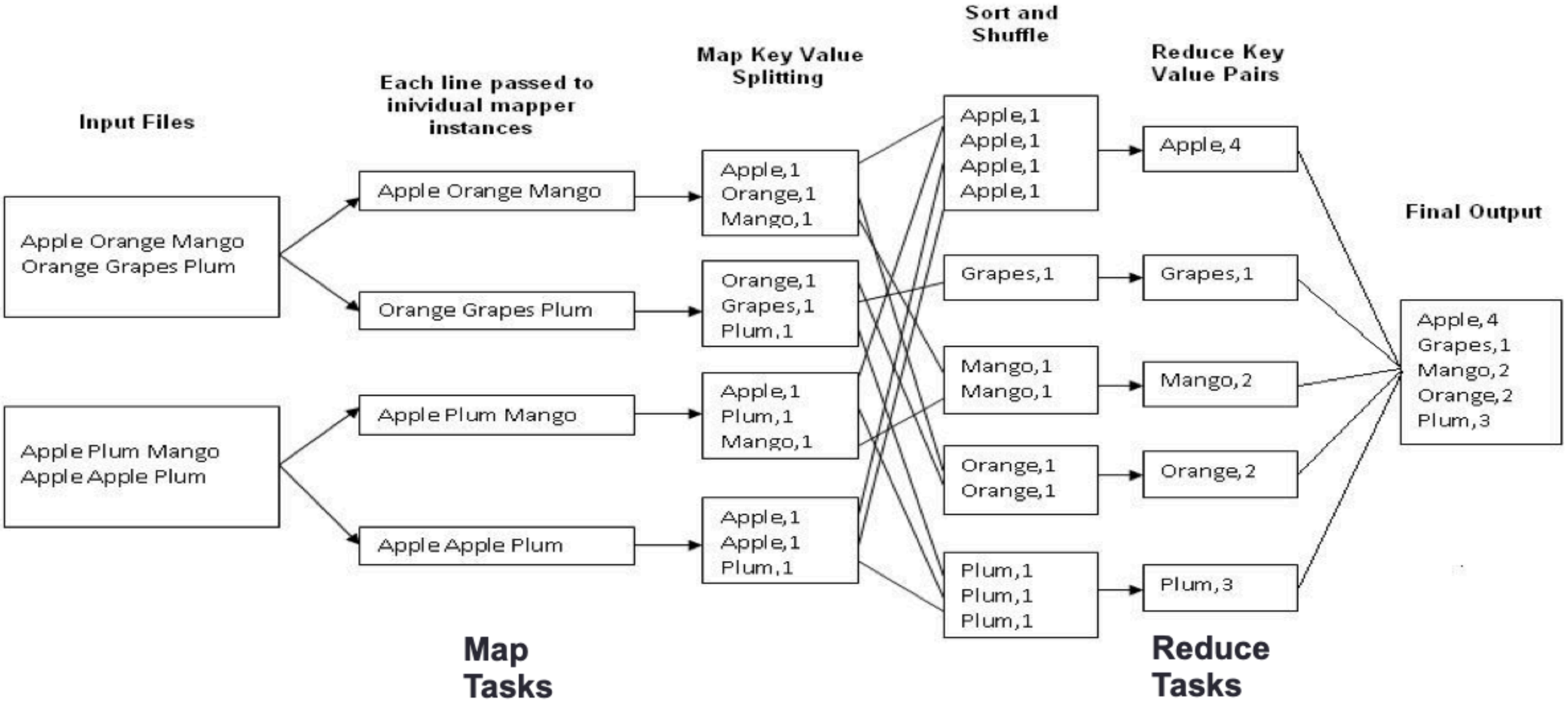
Hadoop - Map-Reduce

- Ou seja, durante o shuffle pode ocorrer movimentação de dados entre os nós para que cada chave seja agrupada em um mesmo nó.
- Essa movimentação de dados é MUITO custosa em alguns casos e afeta diretamente a performance do seu programa.
- Além disso o Hadoop grava resultados intermediários do MapReduce em **DISCO** (final da função map, shuffle sort, e no final da função reduce)



Hadoop - Map-Reduce

Exemplo - Contagem de Palavras



Apache Spark

- Framework de computação distribuída open source.
- In-memory Processing: Diferente do Hadoop, mantém conjuntos de dados na memória RAM entre as etapas de computação, reduzindo custos de I/O de disco.
- Tolerância a Falhas: Utiliza estruturas chamadas RDDs (*Resilient Distributed Datasets*) para reconstruir automaticamente dados perdidos em caso de falha de um nó do cluster.
- Arquitetura Master/Worker: É composto por um nó principal (**Driver**) que coordena as tarefas e múltiplos nós de execução (**Executors**) que processam os dados em paralelo.
- Suporte multilinguagem e API: Tem suporte para múltiplas linguagens como Python, SQL e integra outros frameworks como o próprio Hadoop —> pode ser utilizado para várias aplicações.

Módulos do Apache Spark

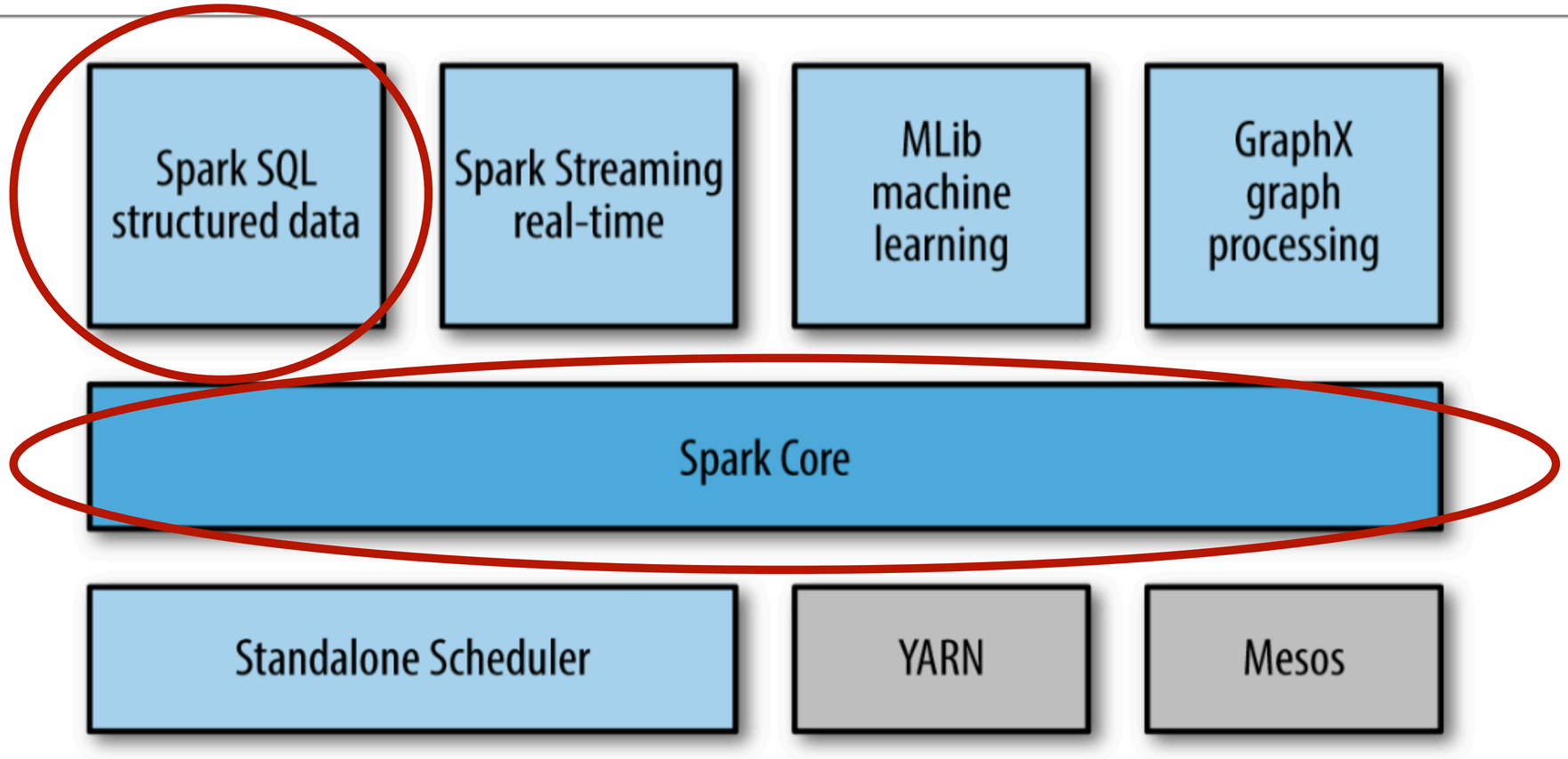


Imagem retirada do livros Learning Spark : https://www.oreilly.com/library/view/learning-spark/9781449359034/

Próximas aulas…

- Apache Spark
  - Funcionamento
  - Introdução rápida a Scala
  - Programação em map-reduce sobre o Spark

Trabalho do Semestre

Implementar dois algoritmos de mineração ou processamento de dados em ambiente paralelo/distribuído e analisar a escalabilidade das soluções conforme o volume de dados aumenta.

- **Divisão:** 5 grupos de 4 pessoas e 1 grupo de 3 pessoas.

Requisitos do Projeto

- **Escolha da Tarefa e Algoritmos:**
  - Escolher **1 tipo de tarefa** da Ciência de Dados (ex.: *clustering*, classificação, regressão, associação, etc.).
  - Implementar **2 algoritmos diferentes** para a tarefa escolhida.
  - **Divisão interna:** Cada dupla do grupo fica responsável pela implementação de 1 dos algoritmos.
- **Linguagem e Tecnologias:**
  - A implementação deve utilizar Java (utilizando Threads, ExecutorService e ForkJoin que vimos) ou em Scala sobre o **Apache Spark, porém** ambos os algoritmos do grupo devem obrigatoriamente usar a mesma linguagem.

Trabalho do Semestre

- **Interface e Padronização:**
  - Os dois algoritmos devem rodar sob a **mesma interface de execução** e gerar o **mesmo formato de saída (*output*)** para garantir uma comparação justa.
- **Datasets e Teste de Escalabilidade:**
  - Selecionar ou gerar **2 datasets** distintos para avaliar o tempo de execução e a escalabilidade dos algoritmos.
  - **Origem dos dados:** Podem ser datasets reais ou gerados sinteticamente (caso sejam sintéticos, devem utilizar distribuições de dados diferentes).
  - **Tamanho dos datasets: 10.000 - 150.000 pontos** (ou pelo menos na casa do 100k)
- **Entregável (Repositório GitHub):**
  - Cada grupo deve criar um repositório no GitHub contendo:
    - Códigos-fonte dos dois algoritmos.
    - Scripts de execução dos testes.
    - **README.md** completo com instruções claras de como compilar e rodar os testes.

Trabalho do Semestre

- Para o grupo de 3 pessoas:

Escolha do Algoritmo: Escolher **1 algoritmo** de Ciência de Dados.

Implementação: Implementar esse algoritmo aplicando **2 decomposições diferentes** de dados ou tarefas (ex.: dois métodos distintos de particionamento de dados).

Padronização: Ambas as decomposições devem rodar sob a **mesma interface de execução** e gerar o **mesmo formato de saída (*output*)**.

Trabalho do Semestre

- Vocês têm liberdade para propor outras tarefas e algoritmos além das sugestões abaixo, porém **dois grupos não podem escolher a mesma combinação de algoritmos**. A confirmação do tema será feita por ordem de envio/registro.
- Tarefas e algoritmos recomendados:
- Tarefa: Clustering ; Algoritmos: k-means e DBSCAN
- Tarefa: Clustering 2; Algoritmos: Hierárquico e k-Medoids
- Tarefa: Mineração de regras de associação ; Algoritmos: Apriori e FP-Growth
- Tarefa: Busca k-NN ; Algoritmos: LSH (Locality Sensitive Hashing) e k-NN Força Bruta (neste caso, deve-se implementar um método de particionamento eficiente/inteligente)
- Tarefa: Mineração de texto; Algoritmos: TF-IDF e CountVectorizer
- Tarefa: Supervisionado ; Algoritmos: Random Forest (para o grupo de 3 pessoas)

Trabalho do Semestre

Cronograma de Entregas e Apresentação

- **Até 25/10 — Formulário Inicial (Google Forms no Classroom):**
  - Preenchimento com: integrantes do grupo, tarefa escolhida, algoritmos a serem implementados, divisão das duplas responsáveis por cada algoritmo e link do repositório no GitHub.
- **Até 12/11 — Entrega do Relatório Final:**
  - Submissão do relatório final (template será postado no Classroom).
  - O relatório deve conter:
    - Explicação sobre o algoritmo implementado e as decisões tomadas para o paralelismo/distribuição.
    - Informações sobre os experimentos e resultados das análises de escalabilidade.
    - Discussão sobre os resultados obtidos e sugestões de melhorias/trabalhos futuros.
- **Dia 13/11 — Apresentação Final:**
  - Apresentação de aproximadamente **20 minutos por grupo**, detalhando os algoritmos implementados, as decisões de projeto e os resultados obtidos.
  - **Obs: A P2 será no dia 27/11**

Uso de IA

- O uso de IA não é permitida para a implementação do algoritmo distribuído ou paralelo.
- É permitido para implementar funções de leitura inicial do arquivo (e apenas isso)
- Pode também ser utilizada para gerar os scripts para automatizar os testes de escalabilidade e realizar os plots dos resultados finais.
- O uso de IA será verificado de acordo com o repositório da equipe
- Favor, incluir no README e no relatório se foi utilizada ou não e deixar de forma explícita para o que foi utilizado e quais arquivos do repositório.

Teste de Escalabilidade - Exemplo

- Dado o dataset, deve-se aumentar de pouco a pouco (10k a 10k pontos, por exemplo) o número de dados e marcar o tempo de execução do seu programa a cada ponto.
- Cuidado ao marcar o tempo para não incluir inicializações não realizadas pelo programa implementado, por exemplo tempo de leitura inicial ou inicialização de framework.
- O objetivo é criar **um plot para cada dataset** parecido com o do lado onde, o eixo X é o tamanho do dataset e o eixo Y é o tempo de execução. Em cada plot deve-se ter 2 linhas, cada uma representando um algoritmo implementado pelo grupo.

